opentext™

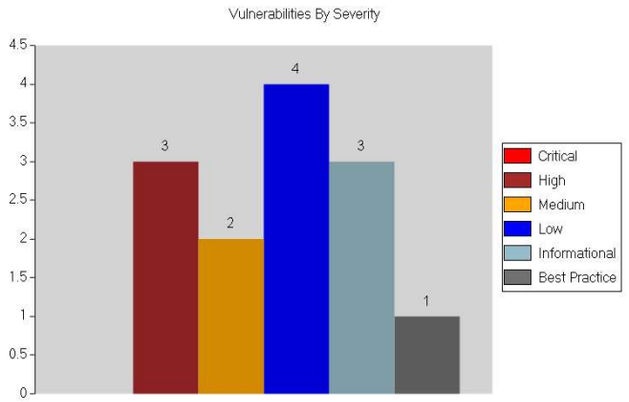
Fortify WebInspect

Vulnerability (Legacy)

Web Application Assessment Report

Scan Name:	Workflow: wsbiomovn1desa.reniec.gob.pe		
Policy:	Standard	**Crawl Sessions:**	15
Scan Date:	7/25/2026 2:35:50 AM	**Vulnerabilities:**	9
Scan Version:	24.4.0.70	**Scan Duration:**	1 Minute : 43 seconds
Scan Type:	Site	**Client:**	Custom

Server: http://wsbiomovn1desa.reniec.gob.pe:7003



High **Insecure Transport**

Summary:

Any area of a web application that possibly contains sensitive information or access to privileged functionality such as remote site administration functionality should utilize SSL or another form of encryption to prevent login information from being sniffed or otherwise intercepted or stolen. http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login has failed this policy. Recommendations include ensuring that sensitive areas of your web application have proper encryption protocols in place to prevent login information and other data that could be helpful to an attacker from being intercepted.

Implication:

An attacker who exploited this design vulnerability would be able to utilize the information to escalate their method of attack, possibly leading to impersonation of a legitimate user, the theft of proprietary data, or execution of actions not intended by the application developers.

Fix:

For Security Operations:
Ensure that sensitive areas of your web application have proper encryption protocols in place to prevent login information and other data that could be helpful to an attacker from being intercepted.

For Development:
Ensure that sensitive areas of your web application have proper encryption protocols in place to prevent login information and other data that could be helpful to an attacker from being intercepted.

For QA:
Test the application not only from the perspective of a normal user, but also from the perspective of a malicious one.

File Names:
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login

High **Often Misused: Login**

Summary:

An unencrypted login form has been discovered. Any area of a web application that possibly contains sensitive information or access to privileged functionality such as remote site administration functionality should utilize SSL or another form of encryption to prevent login information from being sniffed or otherwise intercepted or stolen. If the login form is being served over SSL, the page that the form is being submitted to MUST be accessed over SSL. Every link/URL present on that page (not just the form action) needs to be served over HTTPS. This will prevent Man-in-the-Middle attacks on the login form. Recommendations include ensuring that sensitive areas of your web application have proper encryption protocols in place to prevent login information and other data that could be helpful to an attacker from being intercepted.

Implication:

An attacker who exploited this design vulnerability would be able to utilize the information to escalate their method of attack, possibly leading to impersonation of a legitimate user, the theft of proprietary data, or execution of actions not intended by the application developers.

Fix:

Ensure that sensitive areas of your web application have proper encryption protocols in place to prevent login information and other data that could be helpful to an attacker from being intercepted.

Reference:

Advisory:http://www.kb.cert.org/vuls/id/466433

File Names:
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login

High **Cross-Frame Scripting**

Summary:

A Cross-Frame Scripting (XFS) vulnerability can allow an attacker to load the vulnerable application inside an HTML iframe tag on a malicious page. The attacker could use this weakness to devise a Clickjacking attack to conduct phishing, frame sniffing, social engineering or Cross-Site Request Forgery attacks.

Clickjacking
The goal of a Clickjacking attack is to deceive the victim (user) into interacting with UI elements of the attacker's choice on the target web site without their knowledge and then executing privileged functionality on the victim's behalf. To achieve this goal, the attacker must exploit the XFS vulnerability to load the attack target inside an iframe tag, hide it using Cascading Style Sheets (CSS) and overlay the phishing content on the malicious page. By placing the UI elements on the phishing page so they overlap with those on the page targeted in the attack, the attacker can ensure that the victim must interact with the UI elements on the target page not visible to the victim.

WebInspect has detected a page which potentially handles sensitive information using an HTML form with a password input field and is missing XFS protection.

An effective frame-busting technique was not observed while loading this page inside a frame.
This response while protected by a valid X-Frame-Options does not contain a valid Content-Security-Policy(CSP) frame-ancestors directive. Consider adding CSP as it now replaces X-Frame-Options and is supported in most modern browswers.

Execution:

Create a test page containing an HTML iframe tag whose src attribute is set to http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login. Successful framing of the target page indicates that the application is susceptibile to XFS.

Note that WebInspect will report only one instance of this check across each host within the scope of the scan. The other visible pages on the site may, however, be vulnerable to XFS as well and therefore should be protected against it with an appropriate fix.

Implication:

A Cross-Frame Scripting weakness could allow an attacker to embed the vulnerable application inside an iframe. Exploitation of this weakness could result in:

Hijacking of user events such as keystrokes
Theft of sensitive information
Execution of privileged functionality through combination with Cross-Site Request Forgery attacks

Fix:

The Content Security Policy (CSP) frame-ancestors directive obsoletes the X-Frame-Options header. Both provide for a policy-based mitigation technique against cross-frame scripting vulnerabilities. The difference is that while the X-Frame-Options technique only checks against the top-level document's location, the CSP frame-ancestors header checks for conformity from all ancestors.

If both CSP frame-ancestors and X-Frame-Options headers are present and supported, the CSP directive will prevail. WebInspect recommends using both CSP frame-ancestors and X-Frame-Options headers as CSP is not supported by Internet Explorer and many older versions of other browsers.

In addition, developers must also use client-side frame busting JavaScript as a protection against XFS. This will enable users of older browsers that do not support the X-Frame-Options header to also be protected from Clickjacking attacks.

X-Frame-Options
Developers can use this header to instruct the browser about appropriate actions to perform if their site is included inside an iframe.Developers must set the X-Frame-Options header to one of the following permitted values:

- DENY

Deny all attempts to frame the page
- SAMEORIGIN

The page can be framed by another page only if it belongs to the same origin as the page being framed

Content-Security-Policy: frame-ancestors

Developers can use the CSP header with the frame-ancestors directive, which replaces the X-Frame-Options header, to instruct the browser about appropriate actions to perform if their site is included inside an iframe. Developers can set the frame-ancestors attribute to one of the following permitted values:

- 'none'

Equivalent to "DENY" - deny all attempts to frame the page

- 'self'

Equivalent to "SAMEORIGIN" - the page can be framed by another page only if it belongs to the same origin as the page being framed

- <scheme-source>

Developers can also specify a schema such as http: or https: that can frame the page.

Reference:

Frame Busting:
Busting Frame Busting: A Study of Clickjacking Vulnerabilities on Popular Sites
OWASP: Busting Frame Busting

OWASP:
Clickjacking

Content-Security-Policy (CSP)
CSP: frame-ancestors

Specification:
Content Security Policy Level 2
X-Frame-Options IETF Draft

Server Configuration:
IIS
Apache, nginx

HP 2012 Cyber Security Report
The X-Frame-Options header - a failure to launch

File Names:
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login

Medium **Privacy Violation: Shoulder Surfing**

Summary:

Basic web application security measures include "masking" all passwords to ensure they are not easily visible/recovered. This includes both passwords typed in by the user (which may be viewed by someone looking over the user's shoulder), as well as hidden passwords returned by the server (which may be viewed by looking at the HTML source code in the browser). Recommendations include requiring all password fields in your web application be masked to prevent other users from seeing this information.

Fix:

For Security Operations:
Implement a security policy that prevents display of passwords in clear text.

For Development:
Ensure that passwords entered by users on a site are not displayed in clear text. Also, do not return passwords to the user within hidden form fields.

For QA:
Ensure that passwords entered by users on a site are not displayed in clear text by logging it as a defect and notifying the appropriate developer.

File Names:
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login

Medium **Cross-Frame Scripting**

Summary:

A Cross-Frame Scripting (XFS) vulnerability can allow an attacker to load the vulnerable application inside an HTML iframe tag on a malicious page. The attacker could use this weakness to devise a Clickjacking attack to conduct phishing, frame sniffing, social engineering or Cross-Site Request Forgery attacks.

Clickjacking
The goal of a Clickjacking attack is to deceive the victim (user) into interacting with UI elements of the attacker's choice on the target web site without their knowledge and then executing privileged functionality on the victim's behalf. To achieve this goal, the attacker must exploit the XFS vulnerability to load the attack target inside an iframe tag, hide it using Cascading Style Sheets (CSS) and overlay the phishing content on the malicious page. By placing the UI elements on the phishing page so they overlap with those on the page targeted in the attack, the attacker can ensure that the victim must interact with the UI elements on the target page not visible to the victim.

WebInspect has detected a response containing one or more forms that accept user input but is missing XFS protection.

An effective frame-busting technique was not observed while loading this page inside a frame.
This response while protected by a valid X-Frame-Options does not contain a valid Content-Security-Policy(CSP) frame-ancestors directive. Consider adding CSP as it now replaces X-Frame-Options and is supported in most modern browswers.

Execution:

Create a test page containing an HTML iframe tag whose src attribute is set to http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/proceso. Successful framing of the target page indicates that the application is susceptibile to XFS.

Note that WebInspect will report only one instance of this check across each host within the scope of the scan. The other visible pages on the site may, however, be vulnerable to XFS as well and therefore should be protected against it with an appropriate fix.

Implication:

A Cross-Frame Scripting weakness could allow an attacker to embed the vulnerable application inside an iframe. Exploitation of this weakness could result in:

Hijacking of user events such as keystrokes
Theft of sensitive information
Execution of privileged functionality through combination with Cross-Site Request Forgery attacks

Fix:

The Content Security Policy (CSP) frame-ancestors directive obsoletes the X-Frame-Options header. Both provide for a policy-based mitigation technique against cross-frame scripting vulnerabilities. The difference is that while the X-Frame-Options technique only checks against the top-level document's location, the CSP frame-ancestors header checks for conformity from all ancestors.

If both CSP frame-ancestors and X-Frame-Options headers are present and supported, the CSP directive will prevail. WebInspect recommends using both CSP frame-ancestors and X-Frame-Options headers as CSP is not supported by Internet Explorer and many older versions of other browsers.

In addition, developers must also use client-side frame busting JavaScript as a protection against XFS. This will enable users of older browsers that do not support the X-Frame-Options header to also be protected from Clickjacking attacks.

X-Frame-Options
Developers can use this header to instruct the browser about appropriate actions to perform if their site is included inside an iframe.Developers must set the X-Frame-Options header to one of the following permitted values:

- DENY

Deny all attempts to frame the page

- SAMEORIGIN

The page can be framed by another page only if it belongs to the same origin as the page being framed

Content-Security-Policy: frame-ancestors

Developers can use the CSP header with the frame-ancestors directive, which replaces the X-Frame-Options header, to instruct the browser about appropriate actions to perform if their site is included inside an iframe. Developers can set the frame-ancestors attribute to one of the following permitted values:

- 'none'

Equivalent to "DENY" - deny all attempts to frame the page

- 'self'

Equivalent to "SAMEORIGIN" - the page can be framed by another page only if it belongs to the same origin as the page being framed

- <scheme-source>

Developers can also specify a schema such as http: or https: that can frame the page.

Reference:

Frame Busting:
Busting Frame Busting: A Study of Clickjacking Vulnerabilities on Popular Sites
OWASP: Busting Frame Busting

OWASP:
Clickjacking

Content-Security-Policy (CSP)
CSP: frame-ancestors

Specification:
Content Security Policy Level 2
X-Frame-Options IETF Draft

Server Configuration:
IIS
Apache, nginx

HP 2012 Cyber Security Report
The X-Frame-Options header - a failure to launch

File Names:
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/proceso

Low **Web Server Misconfiguration: Server Error Message**

Summary:

A server error response was detected. The server could be experiencing errors due to a misbehaving application, a misconfiguration, or a malicious value sent during the auditing process. While error responses in and of themselves are not dangerous, per se, the error responses give attackers insight into how the application handles error conditions. Errors that can be remotely triggered by an attacker can also potentially lead to a denial of service attack or other more severe vulnerability. Recommendations include designing and adding consistent error handling mechanisms which are capable of handling any user input to your web application, providing meaningful detail to end-users, and preventing error messages that might provide information useful to an attacker from being displayed.

Implication:

The server has issued a 500 error response. While the body content of the error page may not expose any information about the technical error, the fact that an error occurred is confirmed by the 500 status code. Knowing whether certain inputs trigger a server error can aid or inform an attacker of potential vulnerabilities.

Fix:

For Security Operations:

Server error messages, such as "File Protected Against Access", often reveal more information than intended. For instance, an attacker who receives this message can be relatively certain that file exists, which might give him the information he needs to pursue other leads, or to perform an actual exploit. The following recommendations will help to ensure that a potential attacker is not deriving valuable information from any server error message that is presented.

- Uniform Error Codes: Ensure that you are not inadvertently supplying information to an attacker via the use of inconsistent or "conflicting" error messages. For instance, don't reveal unintended information by utilizing error messages such as Access Denied, which will also let an attacker know that the file he seeks actually exists. Have consistent terminology for files and folders that do exist, do not exist, and which have read access denied.
- Informational Error Messages: Ensure that error messages do not reveal too much information. Complete or partial paths, variable and file names, row and column names in tables, and specific database errors should never be revealed to the end user. Remember, an attacker will gather as much information as possible, and then add pieces of seemingly innocuous information together to craft a method of attack.
- Proper Error Handling: Utilize generic error pages and error handling logic to inform end users of potential problems. Do not provide system information or other data that could be utilized by an attacker when orchestrating an attack.

Removing Detailed Error Messages

Find instructions for turning off detailed error messaging in IIS at this link:

http://support.microsoft.com/kb/294807

For Development:

From a development perspective, the best method of preventing problems from arising from server error messages is to adopt secure programming techniques that prevent problems that might arise from an attacker discovering too much information about the architecture and design of your web application. The following recommendations can be used as a basis for that.

- Stringently define the data type (for instance, a string, an alphanumeric character, etc) that the application will accept.
- Use what is good instead of what is bad. Validate input for improper characters.
- Do not display error messages to the end user that provide information (such as table names) that could be utilized in orchestrating an attack.
- Define the allowed set of characters. For instance, if a field is to receive a number, only let that field accept numbers.
- Define the maximum and minimum data lengths for what the application will accept.
- Specify acceptable numeric ranges for input.

For QA:

The best course of action for QA associates to take is to ensure that the error handling scheme is consistent. Do you receive a different type of error for a file that does not exist as opposed to a file that does? Are phrases like "Permission Denied" utilized which could reveal the existence of a file to an attacker? Inconsistent methods of dealing with errors gives an attacker a very powerful way of gathering information about your web application.

Reference:

Apache:
Security Tips for Server Configuration
Protecting Confidential Documents at Your Site
Securing Apache - Access Control

Microsoft:
How to set required NTFS permissions and user rights for an IIS 5.0 Web server
Default permissions and user rights for IIS 6.0
Description of Microsoft Internet Information Services (IIS) 5.0 and 6.0 status codes

File Names:
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/.%00/WEB-INF/web.xml

---

Low

HTML5: Misconfigured Content Security Policy

Summary:

Content Security Policy (CSP) is an HTTP response header that provides in-depth protection from critical vulnerabilities such as cross-site scripting (XSS) and clickjacking. Inline inclusion of JavaScript in HTML content is considered harmful as a large number of exploited XSS are delivered as inline code. Functions such as eval(), window.setTimeout(), and window.setImmediate() create and execute JavaScript code from strings and are considered dangerous. The CSP header disallows inclusion of inline JavaScript and unsafe eval functions. However, using `unsafe-inline` and `unsafe-eval` values for the script-src directive can bypass that restriction. Carefully consider the use of these values because it significantly weakens the protection provided by the CSP header.

Execution:

Access link http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login through a proxy and notice the `unsafe-inline` and/or `unsafe-eval` in the CSP header in the response. By default, WebInspect flags only one instance of this vulnerability per host because it is typical to set this header at the host level in a server configuration. Perform the following steps to flag all instances of this issue:

- Create a new policy with the selection of checks that you want to include in a rescan. We recommend using the Blank or Passive policy as a base.
- Select this check and uncheck the "FlagAtHost" check input from standard description.
- Save the policy.
- Rescan with this new custom policy.

Implication:

Inline JavaScript code and dynamic evaluation of strings as JavaScript code using functions such as eval(), setTimeout(), and setImmediate() is considered dangerous. A majority of the XSS exploits are delivered and executed using these mechanisms. Disabling the protection provided by the CSP header using `unsafe-inline` and `unsafe-eval` values for the CSP directives can remove a layer of security provided by CSP against these attacks.

Fix:

Remove the unsafe-eval and unsafe-inline values from the CSP directive values.

Reference:

Content Security Policy Level 3
OWASP Content Security Policy
MDN web docs
Content Security Policy (CSP) Quick Reference Guide

File Names:
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login

Low **Cookie Security: Missing SameSite Attribute**

Summary:

The SameSite attribute protects cookies from Cross-Site Request Forgery (CSRF) attacks. The browser automatically appends cookies to every HTTP request made to the site that sets the cookie. Cookies might store sensitive data like session ID and authorization token or site data that is shared between different requests to the same site during a session. An attacker can perform an impersonation attack by generating a request to the authenticated site from a third-party site page loaded on the client machine because the browser automatically appended the cookie to the request.
The SameSite attribute on a cookie allows sites to control that behaviour and prevents browsers from appending the cookie to request if the request is generated from a third-party site page load. The SameSite attribute can have the following three values:

- Strict: When set to Strict, cookies are only sent along with requests upon top level navigation .
- Lax: When set to Lax, cookies are sent with top level navigation from the same host as well as GET requests originated to the host from third-party sites (for example, in iframe, link, href, and so on and the form tag with GET method only).
- None: Cookies are sent in all requests made to the site within the path and domain scope set for the cookie. Requests generated due to form submissions using the POST method are also allowed to send cookies with request.

Please note that cookies that have the SameSite attribute with the value of None must be set with the Secure attribute otherwise the browser rejects the cookies. Additionally, a few specific browser versions reject the SameSite cookie with the None value for example, Chrome versions 51 to 66, versions of the UC Browser on Android prior to version 12.13.2, versions of Safari and embedded browsers on macOS 10.14, and all browsers on iOS 12 reject cookies set with SameSite=None. A suggested workaround for this issue is to set an alternate cookie with a prefix or suffix such as *Legacy*appended to cookiename . Sites can look for this legacy cookie if it does not find a cookie that was set with SameSite=None.

Execution:

Inspect the highlighted cookie value in the HTTP response in the vulnerable session. The cookie is missing the SameSite attribute.

Implication:

Sites that set cookies without the SameSite attribute have an increased risk of CSRF attacks. Using CSRF, an attacker can impersonate a valid user and gain unauthorized access to application functionality. Furthermore, the recent versions of browsers might reject the cookie that is not set with the SameSite attribute.

Fix:

Add the SameSite attribute to all cookies. Starting February 2020, the Chrome browser made SameSite a mandatory attribute for all cookies. Any cookie without the SameSite attribute is either rejected or treated with the default behaviour, which is the same as setting the attribute value to Lax. Therefore, any cookie that must be sent regardless the origination of the request (for example, analytics cookies) should have the SameSite attribute value of none.
Furthermore, we recommend that developers continue to add traditional CSRF mitigations to the site along with SameSite attribute. Many site users might still use older browser versions to access the site. Older browser versions do not understand the SameSite attribute.

Reference:

https://tools.ietf.org/html/draft-ietf-httpbis-rfc6265bis-05
https://www.chromium.org/updates/same-site/incompatible-clients
https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie/SameSite

File Names:
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login

Low **HTML5: Deprecated Header**

Summary:

`X-XSS-Protection` header is enabled. `X-XSS-Protection` refers to a header used by developers to control the browser's behavior and offers protection against cross-site scripting (XSS). The header was designed to filter pages to remove unsafe components or stop loading when cross-site scripting was detected. Internet Explorer 8 introduced the `X-XSS-Protection` header and it was also supported in Edge version 12-16, Chrome version 4-77, Opera version 15-64, and Safari 5-15.3. Modern browsers no longer maintain, support, or enhance implementation of this header.

Having this setting enabled does not necessarily result in a vulnerability, but in some cases such filters are leveraged to perform XSS attacks against users that would otherwise not be possible. The XSS filters in the browser use regular expressions to alter the response. For example, a regex to detect `<script>` tag would alter the tag in the response to `<sc#ipt>`. It would stop the current XSS attack but also have unintended consequences. An attacker could use this knowledge to disable legitimate scripts or client-side scripts containing security features by adding a malicious parameter. By setting the `X-XSS-Protection` to `"1; mode=block"` stops the entire page from loading but it is vulnerable to side-channel attacks [3].

This header can be set explicitly in an application server configuration or in application code. All places should be checked for insecure setting of this header. This check is reported once per host by default and can be made to report all instances by setting check input `"X-XSS-Protection_Aggressive Reporting"`

Execution:

Click the response tab of the highlighted request. You will notice the occurrence of `X-XSS-Protection` header highlighted and value is set to 1.

Implication:

Setting `X-XSS-Protection` header is ineffective for browsers that no longer support this header. Enabling `X-XSS-Protection` header might increase the risk of cross-site scripting attacks against the application if a user accesses it from an older browser that supported this feature.

Fix:

Remove the response header `X-XSS-Protection`.
OpenText recommends using `Content Security Policy (CSP)` to protect against XSS attacks.

Reference:

1 X-XSS-Protection - HTTP MDN
https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-XSS-Protection

2 IE8 XSS Filters
https://media.blackhat.com/bh-eu-10/presentations/Lindsay_Nava/BlackHat-EU-2010-Lindsay-Nava-IE8-XSS-Filters-slides.pdf

3 Abusing Chrome's XSS auditor to steal tokens PortSwigger Research
https://portswigger.net/research/abusing-chromes-xss-auditor-to-steal-tokens

File Names:
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login

Informational **Hidden Field**

Summary:

While preventing display of information on the web page itself, the information submitted via hidden form fields is easily accessible, and could give an attacker valuable information that would prove helpful in escalating his attack methodology. Recommendations include not relying on hidden form fields as a security solution for any area of the web application that contains sensitive information or access to privileged functionality such as remote site administration functionality.

Execution:

Any attacker could bypass a hidden form field security solution by viewing the source code of that particular page.

Implication:

The greatest danger from exploitation of a hidden form field design vulnerability is that the attacker will gain information that will help in orchestrating a far more dangerous attack.

Fix:

Do not rely on hidden form fields as a method of passing sensitive information or maintaining session state. One workable bypass is to encrypt the hidden values in a form, and then decrypt them when that information is to be utilized by a database operation or a script. From a security standpoint, the best method of temporarily storing information required by different forms is to utilize a session cookie.

Whether hidden or not, if your site utilizes values submitted via a form to construct database queries, do not make the assumption that the data is non-malicious. Instead, utilize the following recommendations to sanitize user supplied input.

- Stringently define the data type (for instance, a string, an alphanumeric character, etc) that the application will accept.
- Use what is good instead of what is bad.
- Validate input for improper characters.
- Do not display error messages to the end user that provide information (such as table names) that could be utilized in orchestrating an attack.
- Define the allowed set of characters. For instance, if a field is to receive a number, only let that field accept numbers.
- Define the maximum and minimum data lengths for what the application will accept.
- Specify acceptable numeric ranges for input.

File Names:
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login
- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/proceso

Informational

HLI: Detected Libraries

Summary:

Hacker Level Insights provides developers and security professionals with more context relating to the overall security posture of their application. The **Bootstrap** version **5.3.8** was detected to be in use by during this scan. While these findings do not necessarily represent a security vulnerability, it is important to note that attackers commonly perform reconnaissance of their target in an attempt to identify known weaknesses or patterns. Knowing what the hacker can see provides context which can help teams better secure their applications.

File Names:

- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login

Best Practice

Compliance Failure: Missing Privacy Policy

Summary:

A privacy policy was not supplied by the web application within the scope of this audit. Many legislative initiatives require that organizations place a publicly accessible document within their web application that defines their website's privacy policy. As a general rule, these privacy policies must detail what information an organization collects, the purpose for collecting it, potential avenues of disclosure, and methods for addressing potential grievances.
Various laws governing privacy policies include the Gramm-Leach-Bliley Act, Health Insurance Portability and Accountability Act (HIPAA), the California Online Privacy Protection Act of 2003, European Union's Data Protection Directive and others.

Execution:

All of the web pages accessible within the scope of the scan are sampled for textual content that often constitutes a privacy policy statement. A violation is reported upon completion of the web application crawl without a successful match against any of the web pages.
Note that the privacy policy of your application could be located on another host or within a section of the site that was not configured as part of the scan. To validate, please try to access the privacy policy of your website and check to see if it was part of the scan.

Implication:

Most privacy laws are created to protect residents who are users of the website. Hence, organizations from any part of the world must adhere to these laws if they cater to customers residing in these geographical areas. Failing to do so could result in a lawsuit by the corresponding government against the organization.

Fix:

Declare a comprehensive privacy policy for the website, and ensure that it is accessible from every page that seeks personal information from users. To verify the fix, rescan the site in order to discover and audit the newly added resources.

Descriptions:
Any standard web application privacy policy should include the following components:

- A description of the intended purpose for collecting the data.
- A description of the use of the data.
- Methods for limiting the use and disclosure of the information.
- A list of the types of third parties to whom the information might be disclosed.
- Contact information for inquires and complaints.

Reference:

California Online Privacy Protection Act
http://oag.ca.gov/privacy/COPPA

National Conference of State Legislation
http://www.ncsl.org/issues-research/telecom/state-laws-related-to-internet-privacy.aspx

Gramm-Leach-Bliley Act
http://www.gpo.gov/fdsys/pkg/PLAW-106publ102/pdf/PLAW-106publ102.pdf

Health Insurance Portability and Accountability Act of 1996
https://www.cms.gov/Regulations-and-Guidance/HIPAA-Administrative-Simplification/HIPAAGenInfo/downloads/HIPAALaw.pdf

Health Insurance Portability and Accountability Act of 1996
http://ec.europa.eu/justice/policies/privacy/docs/guide/guide-ukingdom_en.pdf

File Names:

- http://wsbiomovn1desa.reniec.gob.pe:7003/prueba/login
